

**Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники**

Блок задач с Яндекс.Контеста №3
Алгоритмы и Структуры Данных

Выполнил:
Ларионов Владислав Васильевич
Группа Р3209

Проверил:
Косяков Михаил Сергеевич

Санкт-Петербург, 2026

Содержание

I. Машинки	3
Условие:	3
Решение:.....	4
О решении:	5
J. Гоблины и очереди	6
Условие:	6
Решение:.....	7
О решении:	8
K. Менеджер памяти-1	9
Условие:	9
Решение:.....	10
О решении:	12
L. Минимум на отрезке	13
Условие:	13
Решение:.....	14
О решении:	15

I. Машины

Условие:

Петя, которому три года, очень любит играть с машинками. Всего у Пети N различных машинок, которые хранятся на полке шкафа так высоко, что он сам не может до них дотянуться. Одновременно на полу комнаты может находиться не более K машинок. Петя играет с одной из машинок на полу и если он хочет поиграть с другой машинкой, которая также находится на полу, то дотягивается до нее сам. Если же машинка находится на полке, то он обращается за помощью к маме. Мама может достать для Пети машинку с полки и одновременно с этим поставить на полку любую машинку с пола. Мама очень хорошо знает своего ребенка и может предугадать последовательность, в которой Петя захочет играть с машинками. При этом, чтобы не мешать Петиной игре, она хочет совершить как можно меньше операций по подъему машинки с пола, каждый раз правильно выбирая машинку, которую следует убрать на полку. Ваша задача состоит в том, чтобы определить минимальное количество операций. Перед тем, как Петя начал играть, все машинки стоят на полке.

Формат ввода

В первой строке содержатся три числа N , K и P ($1 \leq K, N \leq 100000, 1 \leq P \leq 500000$). В следующих P строках записаны номера машинок в том порядке, в котором Петя захочет играть с ними.

Формат вывода

Выведите единственное число: минимальное количество операций, которое надо совершить Петиной маме.

Пример

Ввод 

```
3 2 7
1
2
3
1
3
1
2
```

Вывод 

```
4
```

Решение:

```
#include <iostream>
#include <set>
#include <vector>

int main() {
    int cars_count, queue_length;
    size_t max_cars_on_floor;
    std::cin >> cars_count >> max_cars_on_floor >> queue_length;

    std::vector<int> cars_queue(queue_length);

    for (int i = 0; i < queue_length; i++) {
        std::cin >> cars_queue[i];
    }

    std::vector<int> next_pos(queue_length);
    std::vector<int> last_pos(cars_count + 1, 1000000000);

    for (int i = queue_length - 1; i >= 0; i--) {
        next_pos[i] = last_pos[cars_queue[i]];
        last_pos[cars_queue[i]] = i;
    }

    int mom_call_count = 0;
    std::set<int> cars_on_floor;
    std::set<std::pair<int, int>> future_cars;
    std::vector<int> next_use_on_floor(cars_count + 1, -1);

    for (int car_index = 0; car_index < queue_length; car_index++) {
        int curr_car = cars_queue[car_index];

        if (cars_on_floor.contains(curr_car)) {
            future_cars.erase({next_use_on_floor[curr_car], curr_car});
            next_use_on_floor[curr_car] = next_pos[car_index];
            future_cars.insert({next_use_on_floor[curr_car], curr_car});
            continue;
        }

        mom_call_count++;

        if (cars_on_floor.size() == max_cars_on_floor) {
            auto it = std::prev(future_cars.end());
            int car_to_remove = it->second;
            cars_on_floor.erase(car_to_remove);
            future_cars.erase(it);
        }

        cars_on_floor.insert(curr_car);
        next_use_on_floor[curr_car] = next_pos[car_index];
        future_cars.insert({next_use_on_floor[curr_car], curr_car});
    }
```

```
std::cout << mom_call_count << std::endl;  
}
```

О решении:

Идея:

Заранее для каждого появления машинки вычисляется также ее следующее появления. Суть в том, чтобы минимизировать количество вызовов мамы, то есть максимизировать количество ситуаций, когда мальчик должен взять машинку уже с пола – аналогия кэша (убираем ту машинку на полку, которая появится через самое долгое время, либо не появится вовсе). Для этого храним множество пар (следующее использование, машинка).

Сложность по времени:

$O(n \log k)$ – так как вычисление следующего появления работает за $O(n)$, а для каждого из n выполняются операции за $O(\log k)$

Сложность по памяти:

$O(n + m)$ – так как массивы очереди и следующего появления занимают $O(n)$, а структуры, связанные с машинками и их состоянием, занимают $O(m)$

Ж. Гоблины и очереди

Условие:

Гоблины Мглистых гор очень любят ходить к своим шаманам. Так как гоблинов много, к шаманам часто образуются очень длинные очереди. А поскольку много гоблинов в одном месте быстро образуют шумную толпу, которая мешает шаманам проводить сложные медицинские манипуляции, последние решили установить некоторые правила касательно порядка в очереди.

Обычные гоблины при посещении шаманов должны вставать в конец очереди. Привилегированные же гоблины, знающие особый пароль, встают ровно в ее середину, причем при нечетной длине очереди они встают сразу за центром.

Так как гоблины также широко известны своим непочтительным отношением ко всяческим правилам и законам, шаманы попросили вас написать программу, которая бы отслеживала порядок гоблинов в очереди.

Формат ввода

В первой строке входных данных записано число N ($1 \leq N \leq 10^5$) — количество запросов. Следующие N строк содержат описание запросов в формате:

- $+ i$ — гоблин с номером i ($1 \leq i \leq N$) встает в конец очереди.
- $* i$ — привилегированный гоблин с номером i встает в середину очереди.
- $-$ — первый гоблин из очереди уходит к шаманам. Гарантируется, что на момент такого запроса очередь не пуста.

Формат вывода

Для каждого запроса типа $-$ программа должна вывести номер гоблина, который должен уйти к шаманам.

Пример 1

Ввод

```
7
+ 1
+ 2
-
+ 3
+ 4
-
-
```

Вывод

```
1
2
3
```

Решение:

```
#include <deque>
#include <iostream>

void balance_deques(std::deque<int>& left_queue, std::deque<int>& right_queue) {
    int goblin_relocant = right_queue.front();
    right_queue.pop_front();
    left_queue.push_back(goblin_relocant);
}

int main() {
    int request_count = 0;
    std::cin >> request_count;

    std::deque<int> left_queue;
    std::deque<int> right_queue;
    for (int i = 0; i < request_count; i++) {
        char request;
        std::cin >> request;

        int goblin_id = 0;
        if (request == '-') {
            int welcome = left_queue.front();
            left_queue.pop_front();
            std::cout << welcome << std::endl;

            if (left_queue.size() < right_queue.size()) {
                balance_deques(left_queue, right_queue);
            }

            continue;
        }

        std::cin >> goblin_id;
        if (request == '+') {
            right_queue.push_back(goblin_id);
            if (left_queue.size() < right_queue.size()) {
                balance_deques(left_queue, right_queue);
            }

            continue;
        }

        if (request == '*') {
            right_queue.push_front(goblin_id);
            if (left_queue.size() < right_queue.size()) {
                balance_deques(left_queue, right_queue);
            }

            continue;
        }
    }
}
```

}

О решении:

Идея:

Очередь разбивается на 2 дека, левый хранит первую половину, правый – вторую. + добавляет гоблина в конец правой очереди, * добавляет гоблина в начало правой очереди, - достает гоблина из начала первой очереди. После каждой операции надо балансировать размеры деков, чтобы крутые гоблины добавлялись ровно в центр.

Сложность по времени:

$O(n)$ – так как выполняется за линию (количество запросов), а все операции с деками за $O(1)$

Сложность по памяти:

$O(n)$ – потому что в двух деках хранятся все добавленные, но еще не удаленные гоблины

К. Менеджер памяти-1

Условие:

Пете поручили написать менеджер памяти для новой стандартной библиотеки языка $\varphi++$. В распоряжении у менеджера находится массив из N последовательных ячеек памяти, пронумерованных от 1 до N . Задача менеджера – обрабатывать запросы приложений на выделение и освобождение памяти. Запрос на выделение памяти имеет один параметр K . Такой запрос означает, что приложение просит выделить ему K последовательных ячеек памяти. Если в распоряжении менеджера есть хотя бы один свободный блок из K последовательных ячеек, то он обязан в ответ на запрос выделить такой блок. При этом непосредственно перед самой первой ячейкой памяти выделяемого блока не должно располагаться свободной ячейки памяти. После этого выделенные ячейки становятся занятыми и не могут быть использованы для выделения памяти, пока не будут освобождены. Если блока из K последовательных свободных ячеек нет, то запрос отклоняется. Запрос на освобождение памяти имеет один параметр T . Такой запрос означает, что менеджер должен освободить память, выделенную ранее при обработке запроса с порядковым номером T . Запросы нумеруются, начиная с единицы. Гарантируется, что запрос с номером T – запрос на выделение, причем к нему еще не применялось освобождение памяти. Освобожденные ячейки могут снова быть использованы для выделения памяти. Если запрос с номером T был отклонен, то текущий запрос на освобождение памяти игнорируется. Требуется написать менеджер памяти, удовлетворяющий приведенным критериям.

Формат ввода

Первая строка входного файла содержит числа N и M – количество ячеек памяти и количество запросов соответственно ($1 \leq N \leq 2^{31} - 1$; $1 \leq M \leq 10^5$). Каждая из следующих M строк содержит по одному числу: $(i+1)$ -я строка входного файла ($1 \leq i \leq M$) содержит либо положительное число K , если i -й запрос – запрос на выделение с параметром K ($1 \leq K \leq N$), либо отрицательное число $-T$, если i -й запрос – запрос на освобождение с параметром T ($1 \leq T < i$).

Формат вывода

Для каждого запроса на выделение памяти выведите в выходной файл результат обработки этого запроса: для успешных запросов выведите номер первой ячейки памяти в выделенном блоке, для отклоненных запросов выведите число -1 . Результаты нужно выводить в порядке следования запросов во входном файле.

Пример 1

Ввод 

```
42 9
7
3
8
-2
6
5
-5
9
4
```

Вывод 

```
1
8
11
19
25
30
19
```

Пример 2

Ввод 

```
128 12
1
2
4
-2
8
-3
16
-5
32
-7
64
-1
```

Вывод 

```
1
2
4
8
16
32
64
```

Решение:

```
#include <iostream>
#include <set>
#include <vector>

int main() {
    long long memory_cells_count, requests_count;
    std::cin >> memory_cells_count >> requests_count;

    std::set<std::pair<long long, long long>> left_right;
    std::set<std::pair<long long, long long>> len_left;
    std::vector<std::pair<long long, long long>> blocks(requests_count + 1, {-1, -1});

    left_right.insert({1, memory_cells_count});
    len_left.insert({memory_cells_count, 1});

    for (long long request_index = 1; request_index <= requests_count; request_index++)
    {
        int request;
        std::cin >> request;
```

```

if (request > 0) {
    auto it = len_left.lower_bound({request, 0});
    if (it == len_left.end()) {
        std::cout << -1 << std::endl;
        continue;
    }

    long long len = it->first;
    long long left = it->second;
    long long right = left + len - 1;
    long long block_right = left + request - 1;

    blocks[request_index] = {left, block_right};

    std::cout << left << std::endl;

    len_left.erase(it);
    left_right.erase({left, right});

    if (block_right < right) {
        long long new_left = block_right + 1;
        left_right.insert({new_left, right});
        len_left.insert({right - new_left + 1, new_left});
    }

    continue;
}

long long block_return_index = -request;
if (blocks[block_return_index].first == -1) {
    continue;
}

long long left = blocks[block_return_index].first;
long long right = blocks[block_return_index].second;
long long new_left = left;
long long new_right = right;

auto it = left_right.lower_bound({left, 0});
if (it != left_right.end() && it->first == right + 1) {
    new_right = it->second;

    len_left.erase({it->second - it->first + 1, it->first});
    left_right.erase(it);
}

it = left_right.lower_bound({left, 0});
if (it != left_right.begin()) {
    auto prev_it = std::prev(it);
    if (prev_it->second == left - 1) {
        new_left = prev_it->first;

        len_left.erase({prev_it->second - prev_it->first + 1, prev_it->first});
    }
}

```

```

        left_right.erase(prev_it);
    }
}

left_right.insert({new_left, new_right});
len_left.insert({new_right - new_left + 1, new_left});
blocks[block_return_index] = {-1, -1};
}
}

```

О решении:

Идея:

Так как мы не можем хранить массив ячеек из-за размера N , будем хранить отрезки. Одновременно хранится множество с границами формата (левая, правая), упорядоченное по левой границе, а также множество формата (размер, левая граница), упорядоченное по размеру. Первое множество нужно для склеивания отрезков памяти, второе для быстрого поиска отрезка минимально свободной длины. При запросе памяти отрезается нужный блок памяти, при возврате приклеивается.

Сложность по времени:

$O(n \log n)$ – так как для каждого из n возможных блоков выполняется константное число операций, каждая из которых $O(\log n)$

Сложность по памяти:

$O(n)$ – так как хранится информация о запросах и о текущем разбиении памяти на блоки

L. Минимум на отрезке

Условие:

Рассмотрим последовательность целых чисел длины N . По ней с шагом 1 двигается «окно» длины K , то есть сначала в «окне» видно первые K чисел, на следующем шаге в «окне» уже будут находиться K чисел, начиная со второго, и так далее до конца последовательности. Требуется для каждого положения «окна» определить минимум в нём.

Формат ввода

В первой строке входных данных содержатся два числа N и K ($1 \leq N \leq 150000, 1 \leq K \leq 10000, K \leq N$) – длины последовательности и «окна», соответственно. На следующей строке находятся N чисел – сама последовательность. Числа последовательности не превосходят по модулю 10^5 .

Формат вывода

Выходные данные должны содержать $N - K + 1$ строк – минимумы для каждого положения «окна».

Пример

Ввод 

```
7 3
1 3 2 4 5 3 1
```

Вывод 

```
1 2 2 3 1
```

Решение:

```
#include <deque>
#include <iostream>

int main() {
    int len, window_size;
    std::cin >> len >> window_size;

    std::deque<std::pair<int, int>> window;
    for (int i = 0; i < len; i++) {
        int curr_elem;
        std::cin >> curr_elem;

        while (window.size() != 0 && window.back().first >= curr_elem) {
            window.pop_back();
        }

        window.push_back({curr_elem, i});

        while (window.size() != 0 && window.front().second <= i - window_size) {
            window.pop_front();
        }

        if (i >= window_size - 1) {
            std::cout << window.front().first << " ";
        }
    }
}
```

О решении:

Идея:

Используем дек для поддержания минимума в текущем окне. В деке храним пары (значение, индекс) в порядке возрастания значений. При добавлении каждого нового элемента из конца удаляются все превосходящие элементы, а из начала удаляются элементы, которые больше не входят в окно. После этого минимум текущего окна всегда находится в начале дека.

Сложность по времени:

$O(n)$ – так как программа работает за линейно

Сложность по памяти:

$O(k)$ – так как в деке храним максимум все значения окна